

PROFESSIONAL PYTHON WORKFLOW: VIRTUAL ENVIRONMENTS

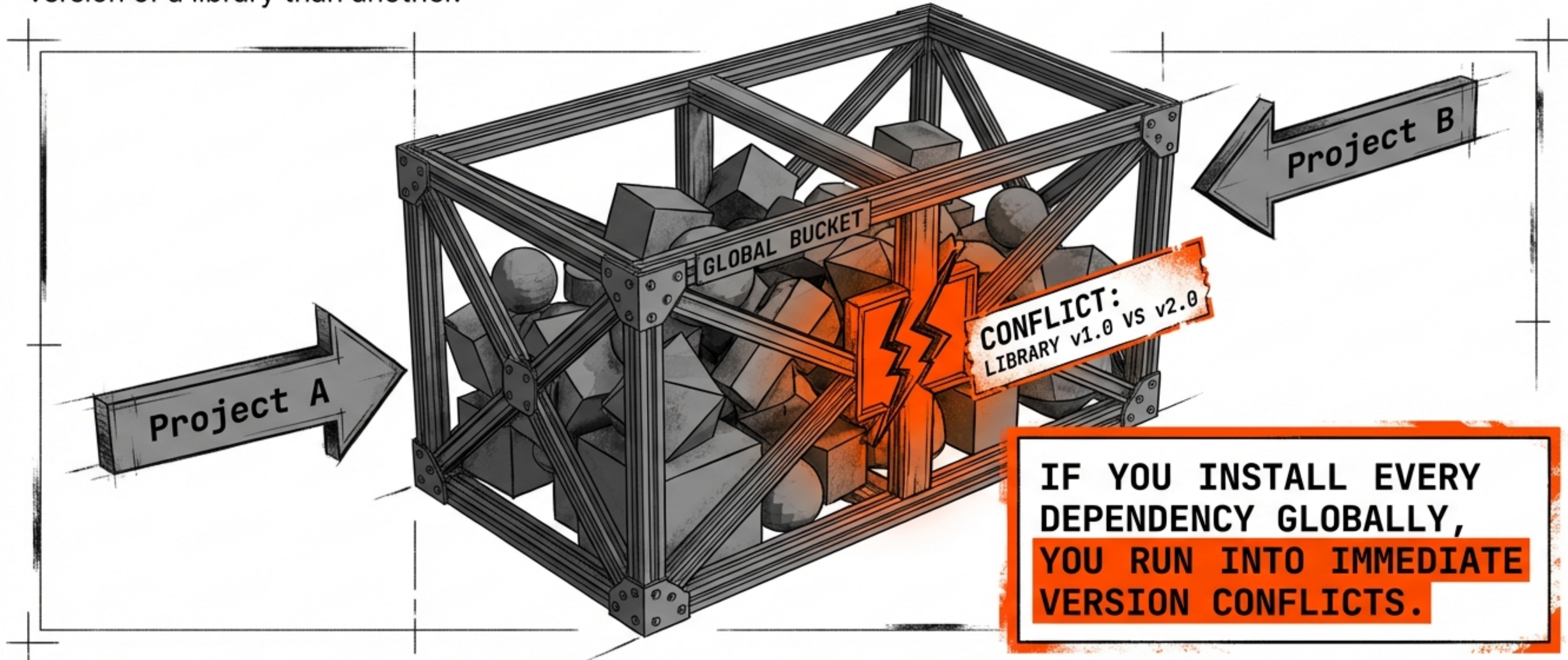
Isolation, Management, and Distribution of Dependencies.



A definitive guide to the `venv` module.

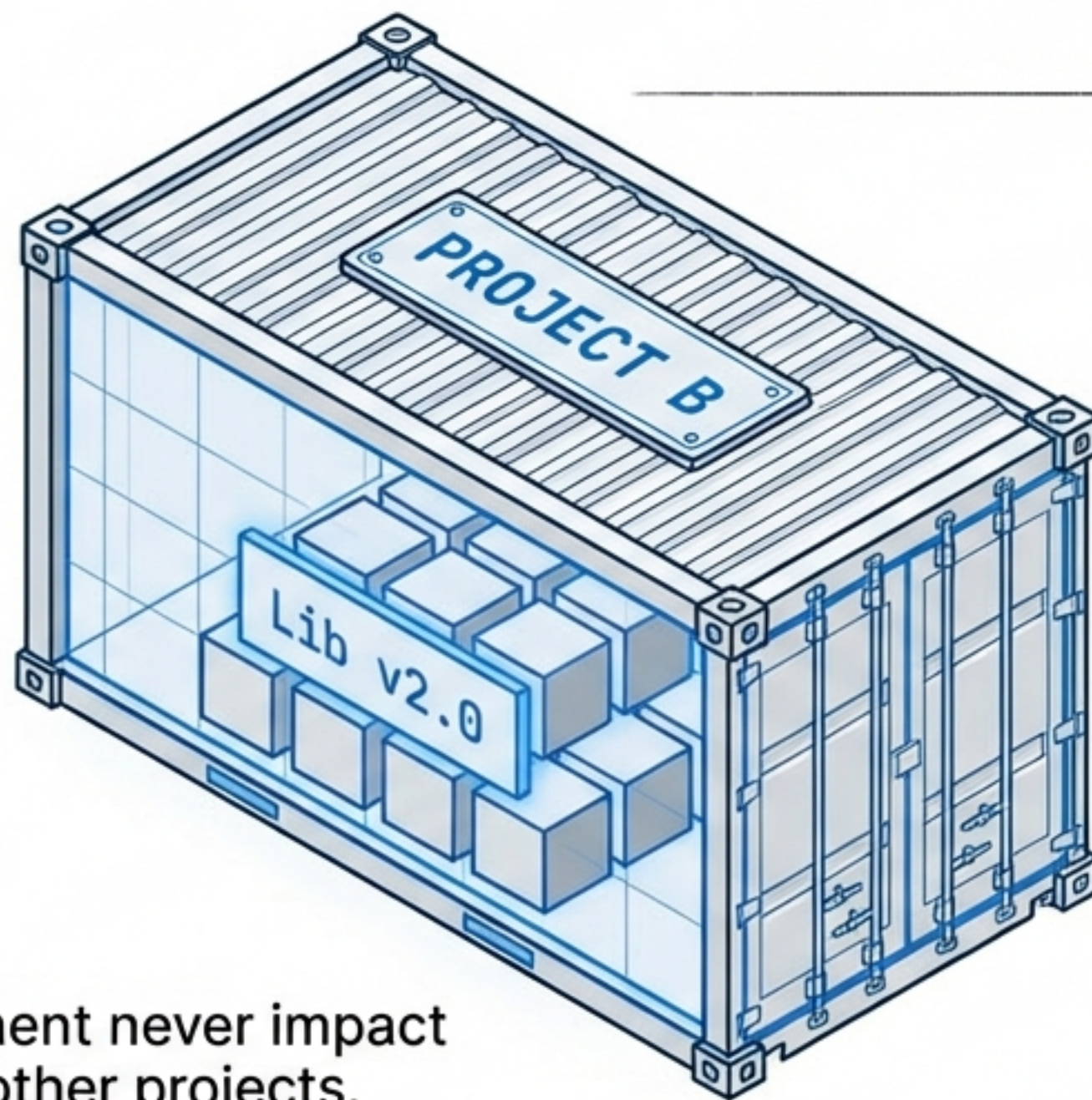
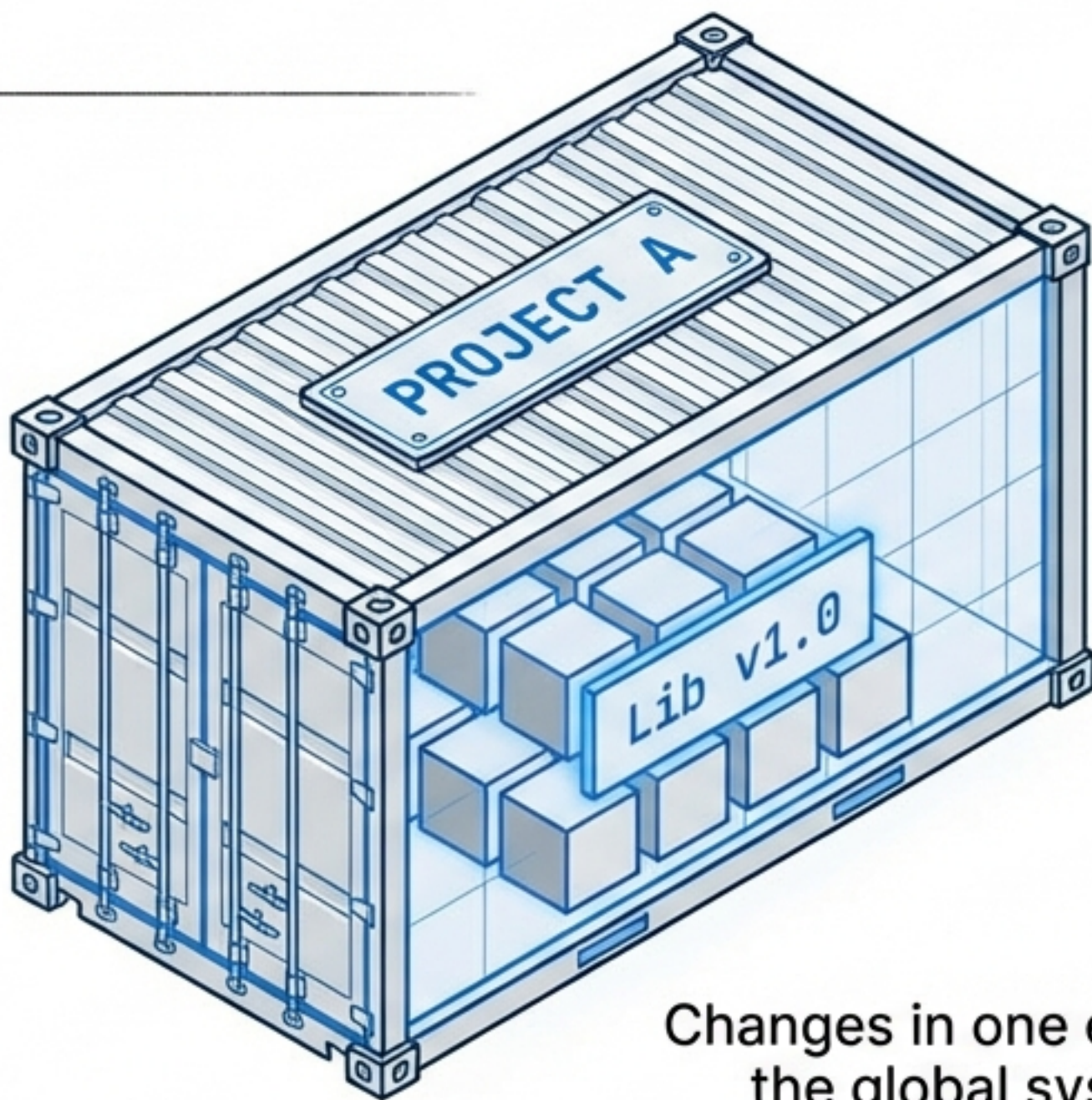
The Trap of Global Installations

Without virtual environments, every dependency is installed into a single global bucket. This creates immediate compatibility issues where one project needs a different version of a library than another.



The Solution: Total Isolation

A virtual environment is a self-contained directory on your system. It maintains a separate, isolated environment for a specific Python project.



Changes in one environment never impact the global system or other projects.

Phase 1: Inception

Creating the Environment

Navigate to your project directory. `venv` is built directly into Python—no external tools required.



```
C:\Project> python -m venv env
```



```
user@mac:~/project$ python3 -m venv env
```

This argument names the folder.
'env' or 'venv' is standard
convention.

Phase 1: Activation

Entering the Sandbox

Creating the folder is not enough; you must explicitly 'enter' the environment to isolate your commands.

Windows



```
env\Scripts\activate.bat
```

Note: The directory is named 'Scripts'.

Mac / Linux



```
source env/bin/activate
```

Note: The directory is named 'bin'.

Verification

How do you know you are safe?



```
(env) User/Project/Path >_
```

Once activated, you will see a parenthesised version of your environment name `(env)` before your normal terminal line. Any command run now applies *only* to this isolated container.

Phase 2: The Clean Slate

Verifying isolation with `pip list`

Global (Deactivated)

```
> pip list
```

Package	Version
-----	-----
numpy	1.26.4
pandas	2.2.1
boto3	1.34.62
scipy	1.12.0
django	5.0.3
requests	2.31.0
matplotlib	3.8.3
scikit-learn	1.4.1

Virtual (Activated)

```
(env) > pip list
```

Package	Version
-----	-----
pip	23.3.1
setuptools	68.2.2

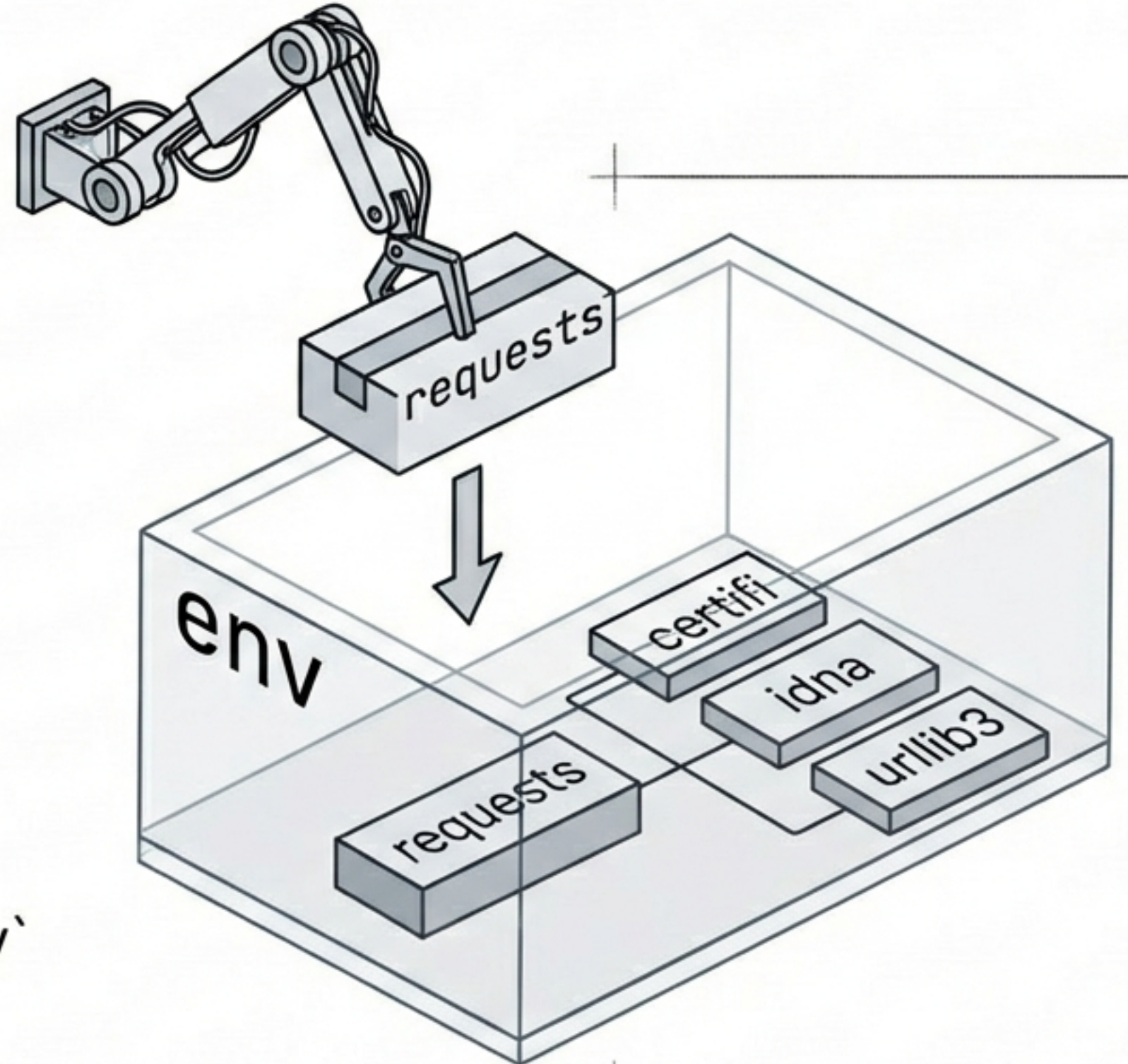
Inside the environment, your global packages do not exist. You are starting from zero.

Installation

Adding specific ingredients

```
pip install requests
```

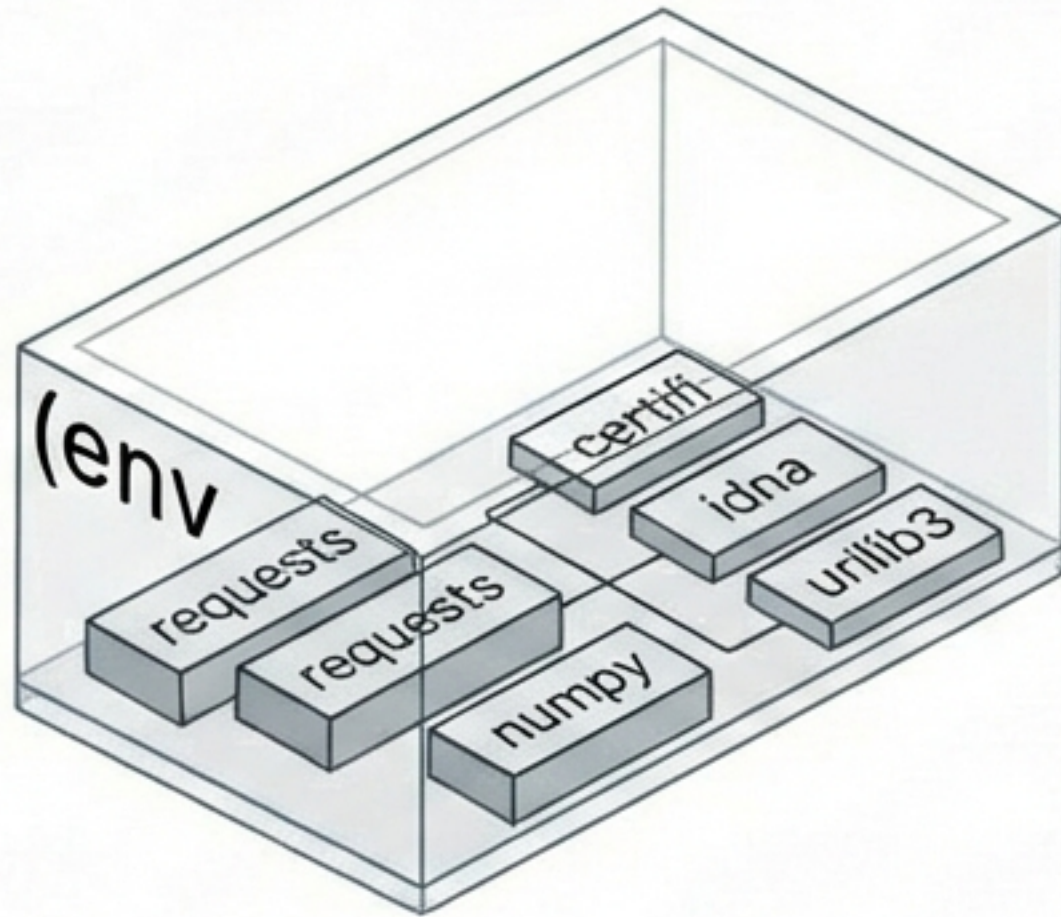
PIP fetches the package and its specific dependencies. These files exist *only* in the `env` folder, not in the global Python installation.



Phase 3: Distribution

Freezing the State

How does another developer know which libraries you used? You must export the dependency list.



```
pip freeze > requirements.txt
```

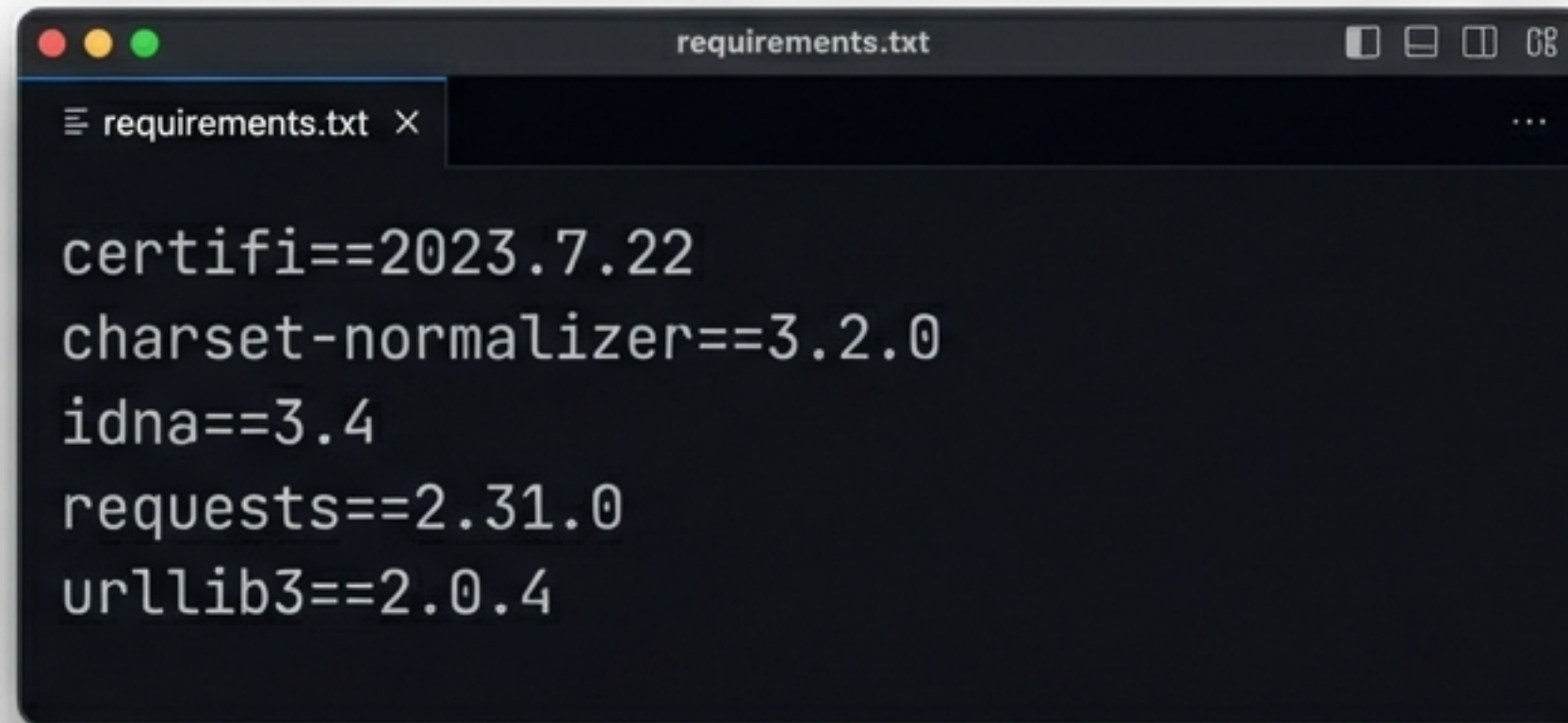


requirements.txt

- **pip freeze**: Lists all packages and version numbers.
- **>**: Redirects output to a file.
- **requirements.txt**: The standard manifesto file.

The Artifact: requirements.txt

This file is the 'recipe' for your project.



```
certifi==2023.7.22
charset-normalizer==3.2.0
idna==3.4
requests==2.31.0
urllib3==2.0.4
```

This file is the 'recipe' for your project. It contains the output of **`pip list`** formatted for machine reading. When you download a project from GitHub, this file tells you exactly what is needed to make the code run.

Replication

Installing from the Recipe

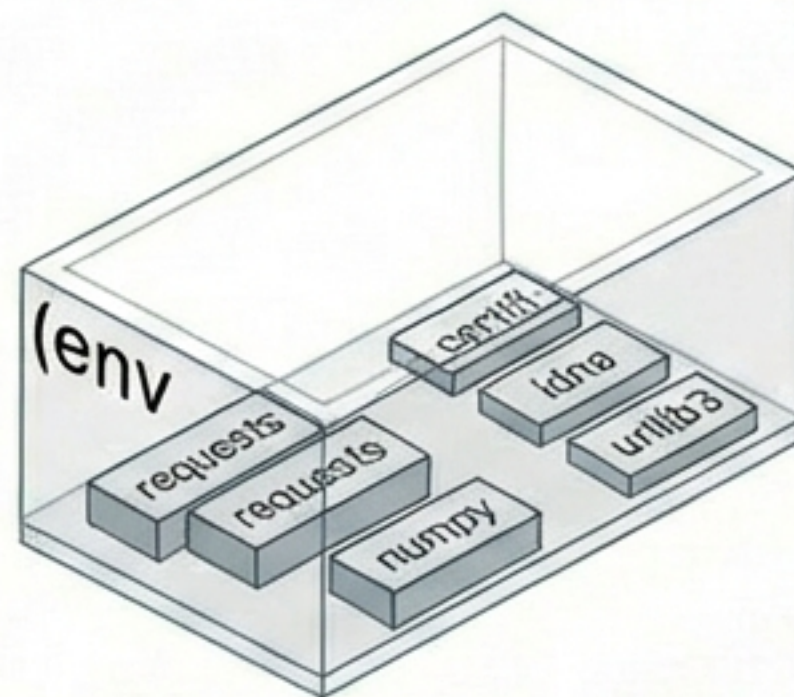
Developer B has downloaded the code and created a fresh, empty environment.

```
(env) $ pip install -r requirements.txt
```



requirements.txt

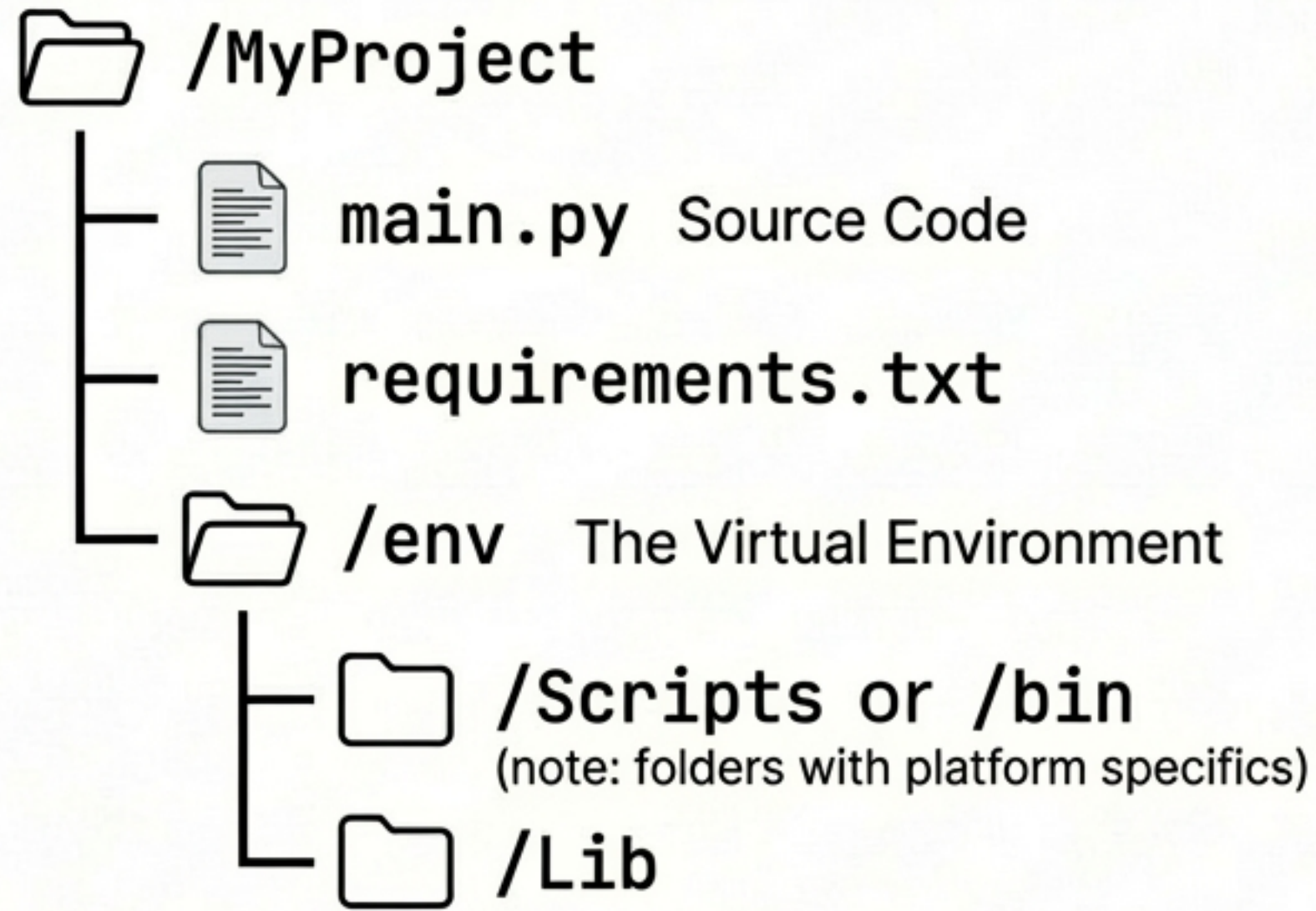
`pip install -r`



Two developers on different machines now have identical environments.

Phase 4: Best Practices

Folder Architecture



 DO	Create the virtual environment <i>inside</i> the project directory, alongside your .py files.
 DON'T	Never save your source code inside the `env` folder. That folder is purely for disposable dependencies.

Termination

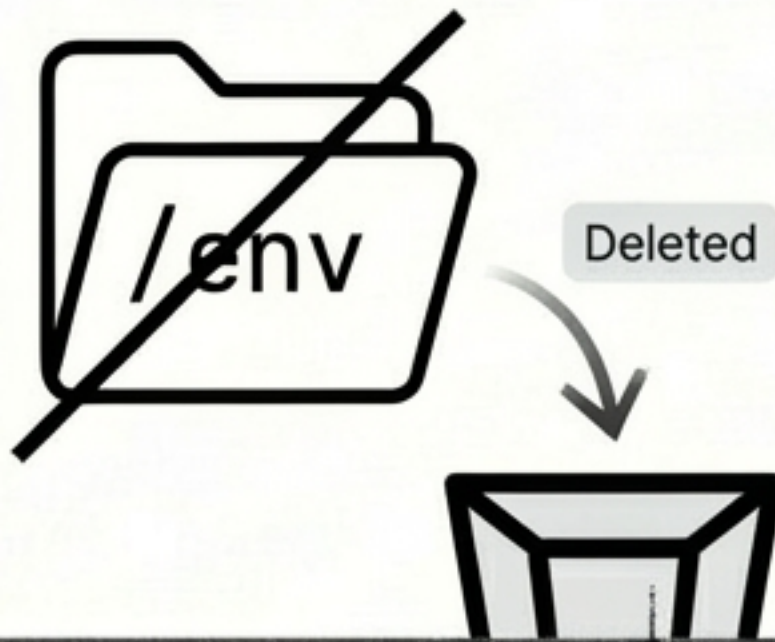
Exiting and Cleaning Up

Exiting

```
deactivate
```

```
deactivate
```

Works on Windows, Mac, and Linux. Returns you to the global shell.



Disposal

There is no “uninstall” command. Because the environment is just a directory, simply deleting the `env` folder removes the environment entirely.

The Developer's Cheat Sheet

Action	Command
Create	<code>python -m venv env (Win) / python3 -m venv env (Mac/Lin)</code>
Activate	<code>env\Scripts\activate (Win) / source env/bin/activate (Mac/Lin)</code>
Install	<code>pip install package_name</code>
Freeze	<code>pip freeze > requirements.txt</code>
Restore	<code>pip install -r requirements.txt</code>
Deactivate	<code>deactivate</code>

Footer Note: Pro Tip: Always check for the `(env)` prefix before installing packages.